Matrix Factorizations & Eigenvalue Algorithms — 80/20 Interview Notes

0. What you should know for an interview

Topic	Priority	What you should be able to do
Eigenvalue / eigenvector	🔴 P0	Define, calculate for a small matrix, explain intuition
Dominant eigenvalue/eigenvector	🔴 P0	Explain what "dominant" means
Power iteration	🔴 P0	Explain algorithm + implement NumPy version
Convergence of power iteration	🔴 P0	Explain why largest eigenvalue dominates
Orthogonal projection	🔴 P0	Understand and implement
Deflation / removing previous eigenvectors	🟠 P1	Explain how additional eigenvectors can be found
QR decomposition	🔴 P0	Explain $A = QR$, Q orthogonal, R upper triangular
Gram-Schmidt → QR	🔴 P0	Understand the connection and implement
QR algorithm	🔴 P0	Explain how repeated QR iterations approximate eigenvalues
Shifted QR	🟠 P1	Know why shifts improve convergence
Numerical instability	🔴 P0	Explain why textbook algorithms aren't always production algorithms
SVD connection	🔴 P0	Know $A = U\Sigma V^T$ and why it matters in ML
Characteristic polynomial	🟡 P2	Understand conceptually; don't rely on it computationally

---

1. Eigenvalues and Eigenvectors

Definition

For a square matrix A, a non-zero vector u is an **eigenvector** if:

$$Au = \lambda u$$

where:

- u= eigenvector
- λ= corresponding eigenvalue
- A= matrix/linear transformation

Intuition

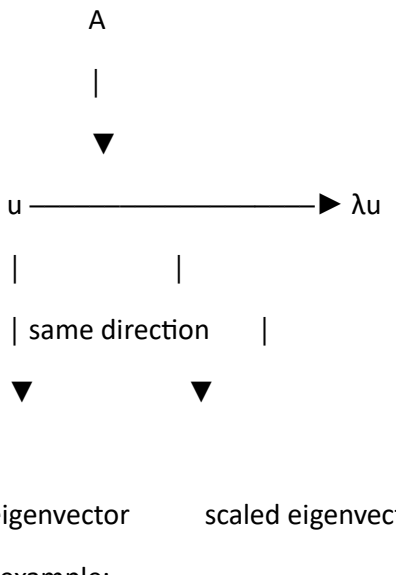
Normally, multiplying a vector by a matrix can:

- rotate it
- stretch it
- shrink it
- change its direction

An eigenvector is a special direction where the matrix **doesn't change the direction**.

It only scales the vector.

Diagram



For example:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Take:

$$u_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Then:

$$Au_1 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

Therefore:

$$Au_1 = 3u_1$$

So:

$$\boxed{\lambda_1 = 3}$$

and

$$\boxed{u_1 = (1, 1)}$$

---

2. Finding Eigenvalues

From:

$$Au = \lambda u$$

we get:

$$Au - \lambda u = 0$$
$$(A - \lambda I)u = 0$$

For a non-zero $u$to exist:

$$\boxed{\det(A - \lambda I) = 0}$$

This is called the **characteristic equation**.

For:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

we have:

$$A - \lambda I = \begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix}$$

Therefore:

$$(2-\lambda)^2 - 1 = 0$$
$$\lambda^2 - 4\lambda + 3 = 0$$
$$(\lambda - 3)(\lambda - 1) = 0$$

Hence:

$$\boxed{\lambda_1 = 3,\ \lambda_2 = 1}$$

Interview point

You should know this mathematically, but:

We generally don't calculate eigenvalues of large matrices by explicitly solving the characteristic polynomial.

Why?

Because computing polynomial roots becomes computationally expensive and numerically problematic for large matrices.

That's why algorithms such as **QR** are used.

---

3. Dominant Eigenvalue

The **dominant eigenvalue** is the eigenvalue with the largest magnitude:

$$|\ \lambda_1\ | > |\ \lambda_i\ |$$

for the other eigenvalues.

For:

$$\lambda_1 = 3,\ \lambda_2 = 1$$

3 is dominant.

Why is this important?

Because repeated multiplication by Aamplifies the component corresponding to the dominant eigenvalue fastest.

This is the fundamental idea behind **power iteration**.

---

4. Power Iteration ⭐⭐⭐

This is one of the most important concepts in your material.

Definition

Power iteration is an iterative algorithm for approximating the dominant eigenvector of a matrix.

Start with some non-zero vector x_0:

$$x_1 = Ax_0$$
$$x_2 = Ax_1 = A^2x_0$$
$$x_3 = A^3x_0$$

and so on.

After repeatedly multiplying by A, normalize the vector.

Eventually:

$$x_k \to u_1$$

where $u_1$is the dominant eigenvector, under suitable convergence conditions.

---

5. Why Power Iteration Works

Suppose the eigenvectors form a basis.

Represent the initial vector as:

$$x_0 = c_1u_1 + c_2u_2 + \cdots + c_nu_n$$

Apply A:

$$Ax_0 = c_1\lambda_1u_1 + c_2\lambda_2u_2 + \cdots + c_n\lambda_nu_n$$

Apply Aagain:

$$A^2x_0 = c_1\lambda_1^2u_1 + c_2\lambda_2^2u_2 + \cdots + c_n\lambda_n^2u_n$$

After $k$iterations:

$$A^kx_0 = c_1\lambda_1^ku_1 + c_2\lambda_2^ku_2 + \cdots + c_n\lambda_n^ku_n$$

Factor out λ_1^k:

$$A^k x_0 = \lambda_1^k \left[c_1 u_1 + c_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k u_2 + \cdots \right]$$

If:

$$| \lambda_1 | > | \lambda_2 | \geq \cdots$$

then:

$$\left|\frac{\lambda_i}{\lambda_1}\right|^k \to 0$$

Therefore:

$$A^k x_0 \approx c_1 \lambda_1^k u_1$$

After normalization:

$$\boxed{x_k \to u_1}$$

The key interview explanation

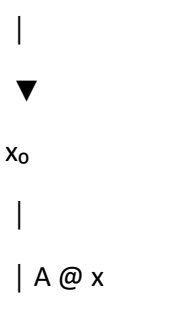
Repeated matrix multiplication amplifies the dominant eigenvector component faster than all other components. Normalization prevents the vector magnitude from exploding, so its direction converges toward the dominant eigenvector.

That is an excellent interview answer.

---

6. Power Iteration Diagram

Initial vector



▼

x_1

|

| normalize

▼

normalized x_1

|

| A @ x

▼

x_2

|

| normalize

▼

normalized x_2

|

▼

...

|

▼

dominant eigenvector

---

7. Power Iteration Implementation

Here is the important implementation from an interview perspective:

```
import numpy as np



def power_iteration(
    A,
```

```
    n_max_steps=1000,
    convergence_threshold=1e-10,
    x_init=None,
    normalize=False
):
    n, m = A.shape

    if n != m:
        raise ValueError("A must be square")

    if x_init is not None:
        x = x_init.reshape(-1, 1)
    else:
        x = np.random.normal(size=(n, 1))

    for _ in range(n_max_steps):

        # Apply matrix transformation
        x_transformed = A @ x

        # Normalize to prevent values from exploding
        x_new = (
            x_transformed /
            np.linalg.norm(x_transformed, ord=np.inf)
        )

        # Measure convergence
        diff = np.linalg.norm(x - x_new)
```

```
        x = x_new

        if diff < convergence_threshold:
            break

    if normalize:
        return x / np.linalg.norm(x)

    return x
```

---

8. Understand Every Important Line

Matrix multiplication

x_transformed = A @ x

This performs:

$$x_{k+1} = Ax_k$$

Normalization

x_new = x_transformed / np.linalg.norm(x_transformed, ord=np.inf)

Why?

Because:

$$A^k x$$

can become extremely large.

For example:

$$3^{100}$$

is huge.

Normalization controls numerical magnitude while preserving the direction.

Convergence

diff = np.linalg.norm(x - x_new)

We're asking:

"Did the direction change significantly compared with the previous iteration?"

Then:

if diff < convergence_threshold:

  break

means:

Stop when the vector has effectively stopped changing.

---

9. Example

Consider:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

The eigenvectors are:

$$u_1 = (1, 1)$$

and

$$u_2 = (1, -1)$$

with eigenvalues:

$$3,\ 1$$

Power iteration converges toward:

$$(1, 1)$$

After unit normalization:

$$\boxed{u_1 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}}$$

because:

$$\sqrt{1^2 + 1^2} = \sqrt{2}$$

so:

$$\frac{1}{\sqrt{2}}(1, 1) = (0.7071, 0.7071)$$

---

10. Recovering the Eigenvalue

Once we have approximately found u:

$$Au \approx \lambda u$$

Therefore:

$$\frac{Au}{u} \approx \lambda$$

component-wise.

For example:

A @ u / u

gives approximately:

[[3.]

[3.]]

So:

$$\boxed{\lambda = 3}$$

A more robust scalar estimate when $u$is normalized is the **Rayleigh quotient**:

$$\boxed{\lambda \approx \frac{u^T Au}{u^T u}}$$

If $u$has unit norm:

$$\lambda \approx u^T A u$$

Interview tip

Know the Rayleigh quotient.

It is much cleaner to say:

"After estimating the eigenvector, I can estimate its eigenvalue using the Rayleigh quotient."

---

11. When Does Power Iteration Fail?

This is extremely important.

Power iteration does **not** universally work.

Condition 1 — Dominant eigenvalue

We generally need:

$$|\lambda_1| > |\lambda_2|$$

If:

$$|\lambda_1| = |\lambda_2|$$

convergence can fail.

Condition 2 — Initial vector must contain the dominant direction

Suppose:

$$x_0 = c_1 u_1 + c_2 u_2$$

We need:

$$c_1 \neq 0$$

If:

$$c_1 = 0$$

then the algorithm has no dominant eigenvector component to amplify.

Condition 3 — Convergence can be slow

If:

$$\frac{|\lambda_2|}{|\lambda_1|}$$

is close to 1, convergence is slow.

Example:

$$\lambda_1 = 10,\ \lambda_2 = 9.9$$

versus:

$$\lambda_1 = 10,\ \lambda_2 = 2$$

The second case converges much faster.

Interview phrase

The convergence rate of power iteration depends strongly on the spectral gap, particularly the ratio $|\lambda_2/\lambda_1|$.

---

12. Finding the Second Eigenvector

Suppose we already found:

$$u_1$$

Power iteration naturally wants to converge toward u_1.

So how do we force it toward u_2?

Remove the u_1 component.

That's where **orthogonal projection** comes in.

---

13. Orthogonal Projection

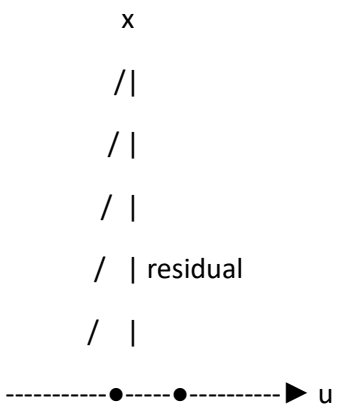
For a unit vector u, projection of x onto u:

$$\boxed{\text{proj}_u(x) = (u^T x)u}$$

More generally:

$$\boxed{\text{proj}_u(x) = \frac{u^T x}{u^T u} u}$$

Diagram



We split:

$$x = \underbrace{\text{proj}_u(x)}_{\text{parallel}} + \underbrace{\left(x - \text{proj}_u(x)\right)}_{\text{orthogonal}}$$

Therefore:

$$\boxed{x_\perp = x - \text{proj}_u(x)}$$

---

14. Projection Matrix

For a vector u:

$$P_u = \frac{uu^T}{u^T u}$$

Then:

$$P_u x$$

gives the projection of $x$onto u.

For a normalized vector:

$$u^T u = 1$$

so:

$$\boxed{P_u = uu^T}$$

---

15. Orthogonal Complement Projection

To remove the $u$component:

$$\boxed{P_\perp = I - \frac{uu^T}{u^T u}}$$

Then:

$$x_\perp = P_\perp x$$

For unit u:

$$\boxed{P_\perp = I - uu^T}$$

Implementation

```
def get_orthogonal_complement_projection(u):
    u = u.reshape(-1, 1)

    n, _ = u.shape

    return (
        np.eye(n)
        - (u @ u.T) /
        np.linalg.norm(u, ord=2) ** 2
    )
```

---

16. Example: Remove the First Eigenvector

Suppose:

$$u_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

and:

$$x = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

The projection onto u_1 is:

$$\frac{x^T u_1}{u_1^T u_1} u_1$$
$$= \frac{2}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Therefore:

$$x_\perp = \begin{bmatrix} 2 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

And:

$$(1, -1)$$

is exactly the second eigenvector direction.

---

17. Generalized Power Iteration

The basic idea:

Find u_1

↓

Remove u_1 component

↓

Power iteration

↓

Find u_2

↓

Remove u_1 and u_2 components

↓

Power iteration

↓

Find u_3

↓

...

For symmetric matrices whose eigenvectors are orthogonal, this can work nicely.

---

18. Finding Multiple Eigenvectors

Conceptually:

```
eigenvectors = []

for i in range(n):

    # Find dominant eigenvector
    ev = power_iteration(A, x_init=x_init)

    # Store it
    eigenvectors.append(ev)

    # Remove its component
    P = get_orthogonal_complement_projection(ev)

    x_init = P @ x_init

    # Normalize
```

```
x_init /= np.linalg.norm(x_init, ord=np.inf)
```

This technique is sometimes described as **deflation through orthogonal projection**.

---

19. Important Numerical Warning

The textbook implementation is excellent for understanding the mathematics.

But:

Don't confuse an educational implementation with a production numerical linear algebra implementation.

Repeated projection and power iteration can accumulate floating-point errors.

For large-scale numerical problems, specialized algorithms and libraries are preferred.

This distinction is an excellent interview point.

---

20. QR Decomposition ⭐⭐⭐

Now we move to the second major concept.

QR decomposition factors a matrix into:

$$\boxed{A = QR}$$

where:

$$\boldsymbol{Q}$$

is an orthogonal matrix:

$$\boxed{Q^T Q = I}$$

$$\boldsymbol{R}$$

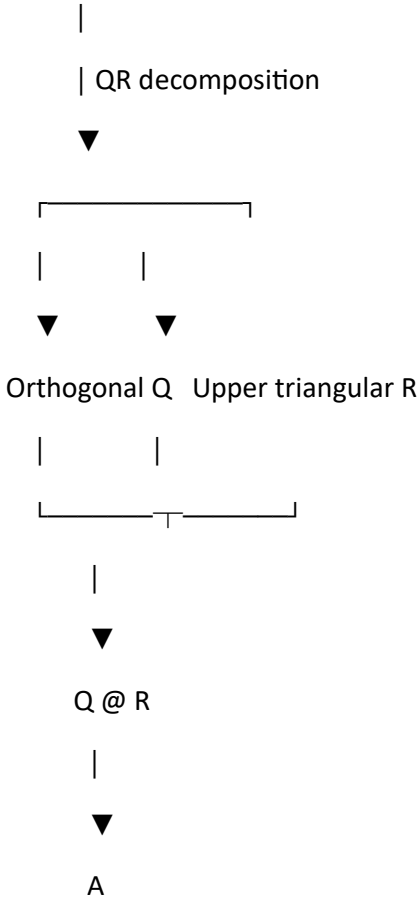
is an upper triangular matrix:

$$R = \begin{bmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{bmatrix}$$

---

21. QR Diagram

Matrix A



---

22. Where Does QR Come From?

QR decomposition comes from **Gram-Schmidt orthogonalization**.

Suppose:

$$A = \begin{bmatrix} | & | & & | \\ v_1 & v_2 & \cdots & v_n \\ | & | & & | \end{bmatrix}$$

We use Gram-Schmidt to convert:

$$v_1, v_2, \ldots, v_n$$

into orthonormal vectors:

$$q_1, q_2, \ldots, q_n$$

Put them into a matrix:

$$Q = [q_1\ q_2\ \cdots\ q_n]$$

The coefficients used to construct the original vectors become R.

Thus:

$$\boxed{A = QR}$$

---

23. Gram-Schmidt 80/20

Given:

$$v_1, v_2$$

First:

$$q_1 = \frac{v_1}{\| v_1 \|}$$

Then remove the $q_1$component from v_2:

$$w_2 = v_2 - (v_2^T q_1)q_1$$

Then normalize:

$$q_2 = \frac{w_2}{\| w_2 \|}$$

Now:

$$q_1^T q_2 = 0$$

and:

$$\| q_1 \| = \| q_2 \| = 1$$

---

24. Simple Gram-Schmidt Example

Take:

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$v_2 = \begin{bmatrix}1\\0\end{bmatrix}$$

Normalize v_1:

$$q_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$$

Projection:

$$(v_2^T q_1)q_1 = \frac{1}{2}\begin{bmatrix}1\\1\end{bmatrix}$$

Remove projection:

$$w_2 = \begin{bmatrix}1\\0\end{bmatrix} - \begin{bmatrix}1/2\\1/2\end{bmatrix} = \begin{bmatrix}1/2\\-1/2\end{bmatrix}$$

Normalize:

$$q_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$$

Therefore:

$$Q = \begin{bmatrix}1/\sqrt{2} & 1/\sqrt{2}\\1/\sqrt{2} & -1/\sqrt{2}\end{bmatrix}$$

---

25. QR Implementation

A simplified educational implementation:

```
def projection_coeff(x, to):
    return np.dot(x, to) / np.dot(to, to)



def projection(x, vectors):
    result = np.zeros_like(x)
```

```
    for v in vectors:
        coeff = projection_coeff(x, v)
        result += coeff * v

    return result


def QR(A):
    n, m = A.shape

    columns = [A[:, i] for i in range(m)]

    q_columns = []
    R = np.zeros((m, m))

    for i, a in enumerate(columns):

        # Remove projections onto previous q vectors
        v = a.copy()

        for j, q in enumerate(q_columns):
            R[j, i] = np.dot(a, q)
            v -= R[j, i] * q

        # Normalize
        R[i, i] = np.linalg.norm(v)

        q = v / R[i, i]
```

```
    q_columns.append(q)

  Q = np.column_stack(q_columns)

  return Q, R
```

---

26. What Should You Check After QR?

Three critical properties:

1. Reconstruction

$$\boxed{A \approx QR}$$

```
np.allclose(A, Q @ R)
```

---

2. Orthogonality

$$\boxed{Q^T Q \approx I}$$

```
np.allclose(Q.T @ Q, np.eye(Q.shape[1]))
```

---

3. Upper triangular R

```
np.allclose(R, np.triu(R))
```

These three checks are excellent interview material.

---

27. Why QR Matters for Eigenvalues ⭐ ⭐ ⭐

This is the key transition.

We want eigenvalues of:

$$A$$

Instead of solving:

$$\det(A - \lambda I) = 0$$

we repeatedly perform QR decompositions.

Start:

$$A_0 = A$$

Compute:

$$A_0 = Q_1 R_1$$

Then reverse the multiplication:

$$\boxed{A_1 = R_1 Q_1}$$

Repeat:

$$A_1 = Q_2 R_2$$

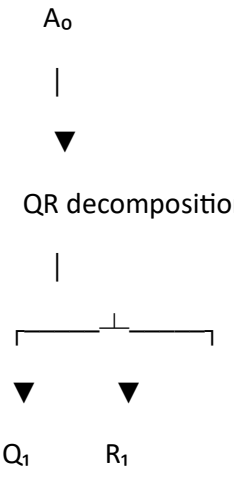
then:

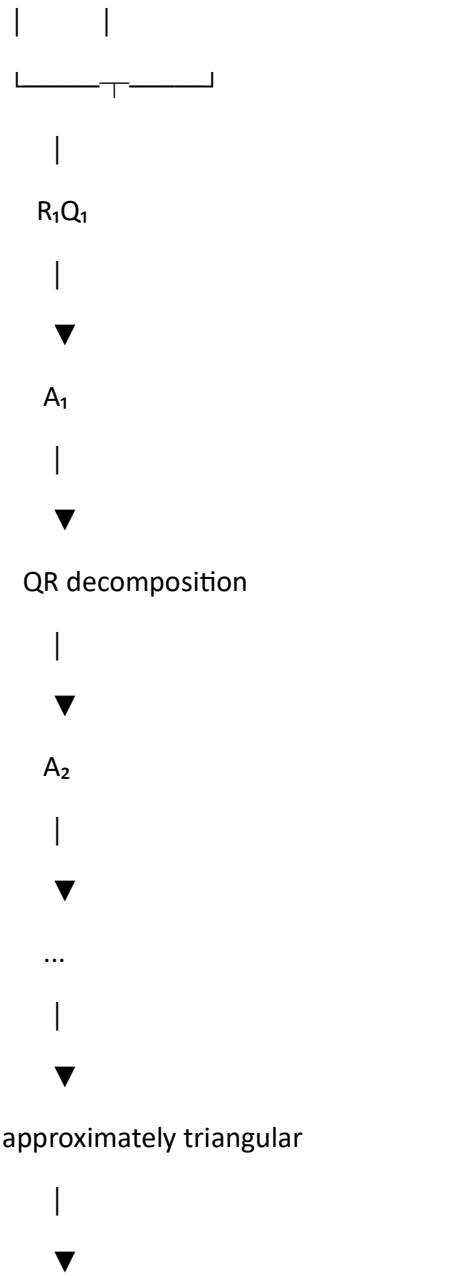
$$A_2 = R_2 Q_2$$

and continue.

---

28. QR Algorithm Diagram





---

29. QR Algorithm Implementation

Educational version:

```
def QR_algorithm(A, n_iter=1000):

    for _ in range(n_iter):
```

Q, R = QR(A)

A = R @ Q

return A

For:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

eventually:

$$A_k \approx \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

Therefore the diagonal contains:

$$\boxed{3,1}$$

which are the eigenvalues.

---

30. The Amazing Property Behind QR

Why does:

$$RQ$$

have the same eigenvalues as:

$$QR?$$

Because if:

$$A = QR$$

then:

$$RQ = Q^T AQ$$

when $Q$is orthogonal.

Since:

$$Q^T Q = I$$

$RQ$is **orthogonally similar** to A.

Similar matrices have the same eigenvalues.

This is a very important interview concept.

---

31. Similar Matrices

Two matrices $A$and $B$are similar if:

$$\boxed{B = P^{-1}AP}$$

for an invertible P.

They represent the same linear transformation under different coordinate systems.

Most importantly:

$$\boxed{A \text{ and } B \text{ have the same eigenvalues}}$$

QR repeatedly transforms the matrix while preserving its eigenvalues.

---

32. Why Does QR Converge?

At a high level, repeated QR transformations gradually move the matrix toward an upper triangular form.

For symmetric matrices, it approaches a diagonal matrix:

$$A_k \to \begin{bmatrix} \lambda_1 & 0 & \cdots \\ 0 & \lambda_2 & \cdots \\ \vdots & & \ddots \end{bmatrix}$$

The diagonal entries are the eigenvalues.

Interview-level answer

QR decomposition creates an orthogonal factorization $A_k = Q_k R_k$. We then form $A_{k+1} = R_k Q_k$, which is similar to A_k, so the eigenvalues are preserved while the matrix progressively approaches triangular form. The diagonal elements then reveal the eigenvalues.

That's the level you should aim for.

---

33. Why Vanilla QR Can Fail

Consider:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Vanilla QR iteration may fail to converge.

The solution is to use a **shift**.

---

34. Shifted QR Algorithm

Instead of decomposing:

$$A_k$$

we shift it:

$$A_k - \alpha_k I$$

Compute:

$$A_k - \alpha_k I = Q_k R_k$$

Then:

$$\boxed{A_{k+1} = R_k Q_k + \alpha_k I}$$

The shift $\alpha_k$is chosen to accelerate convergence.

Common strategies include:

- Rayleigh quotient shift
- Wilkinson shift

You don't need the detailed derivations for most interviews unless you're interviewing for numerical linear algebra/scientific computing.

---

35. Numerical Stability — Very Important

The chapter makes an important engineering distinction:

Power iteration

Pros

- Simple
- Easy to understand
- Easy to implement
- Useful for dominant eigenvector

Cons

- Can converge slowly
- Has convergence conditions
- Can be numerically unstable
- Not appropriate for computing all eigenvalues of arbitrary large matrices

QR

Pros

- General-purpose eigenvalue computation
- Much more practical
- Numerically sophisticated versions are widely used

Cons

- More complicated
- Efficient implementations use optimizations such as shifts and Hessenberg reduction

---

36. SVD Connection ⭐ ⭐ ⭐

The chapter ends by connecting these ideas to SVD.

For a matrix:

$$A \in \mathbb{R}^{m \times n}$$

SVD is:

$$\boxed{A = U\Sigma V^T}$$

where:

- U= orthogonal matrix
- Σ= diagonal matrix of singular values
- V= orthogonal matrix

Diagram

A

|

▼

V^T rotation

|

▼

Σ scaling

|

▼

U rotation

|

▼

Ax

Conceptually:

SVD says a complicated linear transformation can be decomposed into rotation/reflection → scaling → rotation/reflection.

This is one of the most important interpretations in ML.

---

37. Eigenvalues vs Singular Values

Don't confuse these.

Eigenvalue

Defined through:

$$Av = \lambda v$$

Only square matrices have ordinary eigenvalues.

Singular value

Comes from SVD:

$$A = U\Sigma V^T$$

Singular values are non-negative:

$$\sigma_i \geq 0$$

and exist for **any rectangular matrix**.

---

38. Why SVD Matters in Machine Learning

SVD appears in:

- dimensionality reduction
- PCA
- recommendation systems
- low-rank approximation
- noise reduction
- latent semantic analysis
- matrix compression
- numerical optimization

For example, low-rank approximation:

$$A \approx U_k \Sigma_k V_k^T$$

keeps only the largest $k$singular values.

That allows us to represent a large matrix using much less information.

---

39. Power Iteration vs QR vs SVD

Feature	**Power Iteration**	**QR Algorithm**	**SVD**
Main purpose	Dominant eigenvector	Eigenvalues/eigenvectors	Matrix factorization
Difficulty	Low	Medium/High	High
Easy to implement	✅	✅ educationally	More difficult
Large-scale use	Specialized	Very common	Very common
ML relevance	Medium/High	Medium	🔴 Very High
Main idea	Repeated Ax	Repeated QR	Orthogonal + scaling
Main weakness	Convergence/stability	Complexity	Computational cost

---

40. Production Perspective

This is particularly important for your **AI/ML engineering** goal.

You should understand the algorithms, but you generally shouldn't implement numerical linear algebra from scratch in production.

For example:

import numpy as np

eigenvalues, eigenvectors = np.linalg.eig(A)

For symmetric matrices:

eigenvalues, eigenvectors = np.linalg.eigh(A)

And SVD:

U, S, Vt = np.linalg.svd(A)

The important engineering skill is knowing:

What mathematical operation do I need, what algorithm is appropriate, what assumptions does it have, and what numerical behavior should I expect?

---

41. Critical Difference: eig vs eigh

This is an excellent practical interview point.

General matrix

np.linalg.eig(A)

Symmetric/Hermitian matrix

np.linalg.eigh(A)

For a real symmetric matrix:

$$A = A^T$$

you get particularly nice mathematical properties:

- eigenvalues are real
- eigenvectors can be chosen orthonormal
- numerical algorithms can exploit symmetry

Your matrix:

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

is symmetric.

---

42. 80/20 Mental Model

You should be able to remember the whole chapter as:

EIGENVALUES

|

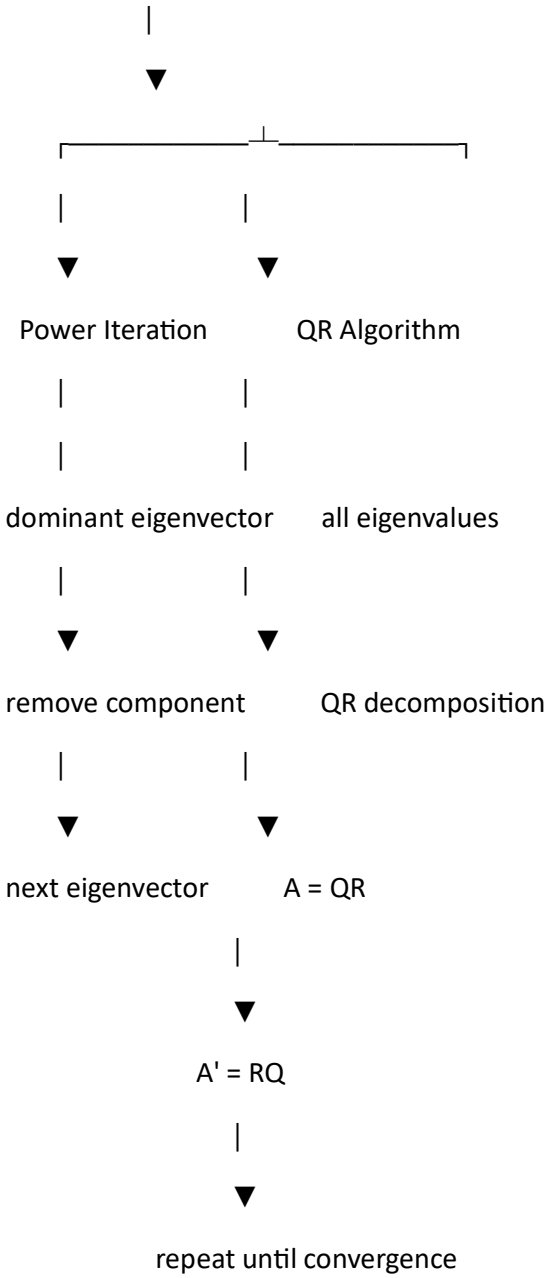
▼

Special directions

|

▼

Au = λu



---

43. Interview Questions You Should Prepare

P0 — Must answer

Q1. What is an eigenvector?

Answer:

An eigenvector is a non-zero vector whose direction remains unchanged when a matrix transformation is applied. Mathematically, $Av = \lambda v$, where $\lambda$is the corresponding eigenvalue.

Q2. What is the dominant eigenvalue?

The eigenvalue with the largest absolute magnitude.

Q3. Explain power iteration.

Start with a non-zero vector, repeatedly multiply it by A, normalize the result, and repeat. If the dominant eigenvalue is unique in magnitude and the initial vector has a non-zero component along its eigenvector, the vector converges toward the dominant eigenvector.

Q4. Why normalize during power iteration?

To prevent numerical overflow or excessively large values while preserving the direction of the vector.

Q5. When can power iteration fail?

Mention:

1. no unique dominant magnitude
2. initial vector has zero dominant component
3. slow convergence when eigenvalues are close
4. numerical issues

Q6. What is QR decomposition?

QR decomposition factors a matrix as $A = QR$, where Qhas orthonormal columns and $R$is upper triangular.

Q7. How is QR related to Gram-Schmidt?

Gram-Schmidt converts the columns of Ainto an orthonormal basis. Those orthonormal vectors form Q, while the projection coefficients form R.

Q8. Explain the QR algorithm.

Repeatedly decompose $A_k = Q_k R_k$, then set $A_{k+1} = R_k Q_k$. The transformation preserves eigenvalues while driving the matrix toward triangular form. The diagonal then approximates the eigenvalues.

Q9. Why use shifted QR?

A shift changes the iteration so that convergence is faster and problematic cases that cause vanilla QR to stagnate can be handled more effectively.

Q10. Why don't we simply solve $\det(A - \lambda I) = 0$?

For small matrices it is useful mathematically, but for large matrices explicitly forming and solving the characteristic polynomial is computationally expensive and numerically undesirable. Practical numerical libraries use iterative matrix algorithms such as QR-based methods.

---

44. P1 — Important Follow-ups

Q11. Why are eigenvectors of symmetric matrices special?

For a real symmetric matrix:

$$A = A^T$$

the spectral theorem tells us that it has an orthonormal eigenbasis.

Therefore:

$$A = U \Lambda U^T$$

where:

$$U^T U = I$$

Q12. What is a projection matrix?

For projection onto vector u:

$$P = \frac{u u^T}{u^T u}$$

and:

$$Px$$

is the projected vector.

Q13. How do you remove an eigenvector component?

Use:

$$\boxed{x_\perp = \left(I - \frac{uu^T}{u^T u}\right)x}$$

Q14. What is a similarity transformation?

$$B = P^{-1}AP$$

It changes the coordinate representation of a linear transformation while preserving eigenvalues.

---

45. P2 — Don't Overinvest Initially

You don't need to spend disproportionate time memorizing:

- the full proof of generalized power iteration
- detailed Wilkinson shift derivation
- complete QR convergence proof
- characteristic polynomial theory for high dimensions
- deriving every QR coefficient by hand

For your current **AI/ML engineering + backend engineering** direction, understand the intuition, mathematical structure, implementation, numerical limitations, and practical library usage.

---

46. One Complete Practical Example

Let's put everything together.

import numpy as np

```
A = np.array([
    [2.0, 1.0],
    [1.0, 2.0]
])

# Power iteration
x = np.random.rand(2, 1)

for _ in range(100):
    x_new = A @ x
    x_new = x_new / np.linalg.norm(x_new)

    if np.linalg.norm(x_new - x) < 1e-10:
        break

    x = x_new

print("Eigenvector:")
print(x)

# Rayleigh quotient
eigenvalue = (
    x.T @ A @ x
) / (
    x.T @ x
)
```

print("Eigenvalue:")

print(eigenvalue)

Expected approximately:

$$u = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

and:

$$\lambda = 3$$

Then using NumPy:

values, vectors = np.linalg.eigh(A)

print(values)

print(vectors)

You get eigenvalues:

$$\boxed{1,\ 3}$$

and corresponding orthonormal eigenvectors.

---

47. Final 80/20 Cheat Sheet

Memorize this:

Eigenpair

$$\boxed{Av = \lambda v}$$

Eigenvalues

$$\boxed{\det(A - \lambda I) = 0}$$

Dominant eigenvalue

$$\boxed{|\ \lambda_1\ | > |\ \lambda_i\ |}$$

Power iteration

$$\boxed{x_{k+1} = \frac{Ax_k}{\| Ax_k \|}}$$

Why it works

$$\boxed{\left|\frac{\lambda_2}{\lambda_1}\right|^k \to 0}$$

Projection onto u

$$\boxed{P_u = \frac{uu^T}{u^T u}}$$

Remove $u$component

$$\boxed{P_\perp = I - \frac{uu^T}{u^T u}}$$

QR decomposition

$$\boxed{A = QR}$$

with:

$$\boxed{Q^T Q = I}$$

and Rupper triangular.

QR iteration

$$\boxed{A_k = Q_k R_k}$$
$$\boxed{A_{k+1} = R_k Q_k}$$

Shifted QR

$$\boxed{A_k - \alpha_k I = Q_k R_k}$$
$$\boxed{A_{k+1} = R_k Q_k + \alpha_k I}$$

SVD

$$\boxed{A = U\Sigma V^T}$$

The one story to remember

Eigenvectors are special directions of a linear transformation. Power iteration repeatedly applies the transformation so the dominant direction wins. To find additional directions, remove previously discovered components using orthogonal projections. QR decomposition comes from Gram-Schmidt and factors a matrix into an orthogonal matrix and an upper-triangular matrix. Repeatedly swapping the QR factors produces the QR algorithm, which preserves eigenvalues while driving the matrix toward triangular/diagonal form. SVD generalizes the decomposition idea to arbitrary rectangular matrices and is especially important in machine learning.

For your interviews, **Power Iteration + Orthogonal Projection + QR Decomposition + QR Algorithm + SVD intuition** are the core 20% to master from this section.